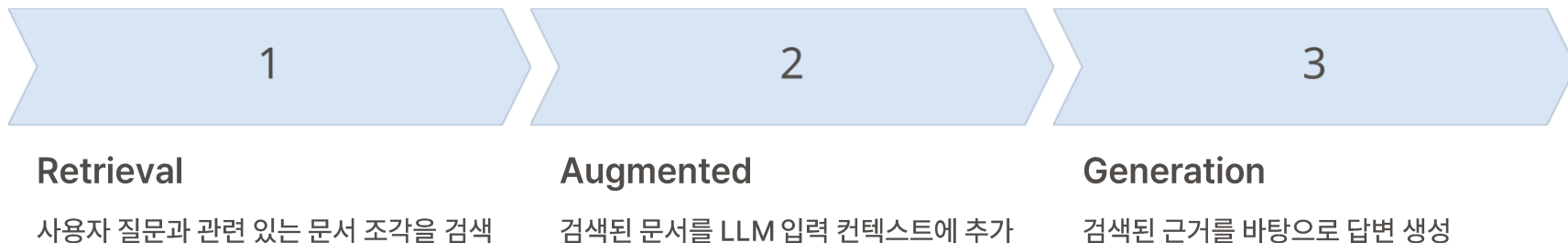


1.RAG 개념 이해: 문서 근거 기반 답변

RAG: 문서 근거 기반 답변 구조

RAG(Retrieval-Augmented Generation)는 LLM이 바로 답변을 생성하는 것이 아니라, 관련 문서를 먼저 검색하고 그 근거를 바탕으로 답변하도록 설계된 아키텍처입니다.



i 제조 업무에서 중요한 것은 그럴듯한 답변이 아닙니다. 어떤 매뉴얼, 품질 기준, 장애 대응 문서를 근거로 답변했는지 추적 가능해야 합니다.

내부 문서 기반 답변을 위한 RAG 구조

LLM은 기본적으로 회사 내부 문서나 최신 업무 문서를 알고 있다고 볼 수 없습니다. 제조 장애 대응에서는 LLM에게 바로 답하게 하기보다, 먼저 관련 문서를 찾아 근거로 전달해야 합니다.

기존 방식의 문제

LLM이 학습 데이터 기반으로 그럴듯한 답변을 생성 →
근거 추적 불가, 신뢰성 낮음

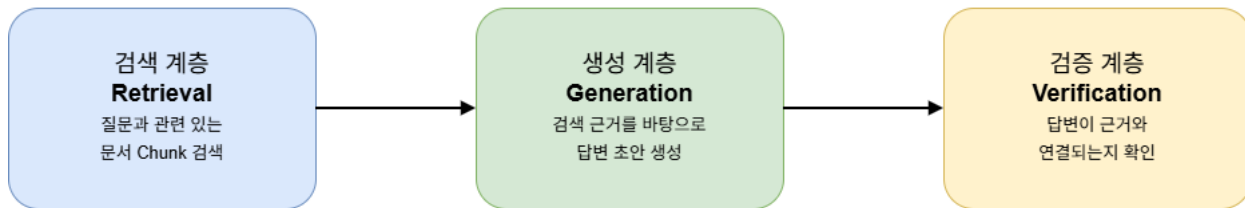
RAG 방식의 핵심

ALM-TEMP-402 알람 발생 시 → 알람 매뉴얼·품질 기준
서를 먼저 검색 → 해당 내용을 근거로 답변 생성

RAG의 핵심은 "답변 생성"보다 먼저 "근거 검색"을 수행
하고, 그 근거를 Agent 실행 흐름 안에 남기는 것입니다.

RAG의 설계적 의미

RAG는 답변 생성 기술이 아니라, 검색·생성·검증 계층을 분리해
근거 기반 답변을 만드는 Agent 설계 구조입니다.



핵심: 답변이 자연스러운가보다, 어떤 문서 근거로 답했는지 확인 가능한가가 중요합니다.

RAG 아키텍처: 검색·생성·검증 계층 분리

RAG를 아키텍처 관점에서 보면, 역할이 명확히 분리된 세 계층으로 구성됩니다. 각 계층을 분리함으로써 검색 품질, 답변 품질, 근거 추적을 독립적으로 점검할 수 있습니다.

검색 계층 (Retrieval Layer)

사용자 질문과 관련 있는 문서 조각(chunk)을 찾습니다. 검색 결과가 부족할 경우 질문을 재작성하거나 검색 범위를 조정합니다.

답변 생성 계층 (Generation Layer)

검색된 문서 조각을 근거로 답변 초안을 생성합니다. 근거 없이 단정하지 않고, 추가 확인이 필요한 경우 명시합니다.

검증 계층 (Verification Layer)

생성된 답변이 검색 근거를 충분히 반영했는지 확인합니다. Grounding 검증을 통해 답변 품질과 근거 추적 가능성을 관리합니다.

❑ RAG는 단순히 문서를 붙여 답변하는 기능이 아니라, 검색·생성·검증을 분리해 Agent 답변의 근거와 품질을 관리하는 구조입니다.

제조 업무에서 RAG가 필요한 이유

제조 장애 대응에서는 하나의 질문에 여러 문서가 연결됩니다. 제조회 RAG Agent는 서로 다른 문서를 함께 검색하고, 그 결과를 업무 판단 근거로 연결하는 구조가 필요합니다.

알람 매뉴얼

알람 코드의 의미와 1차 확인 항목

장애 대응 가이드

증상별 원인 후보와 점검 절차

품질 기준서

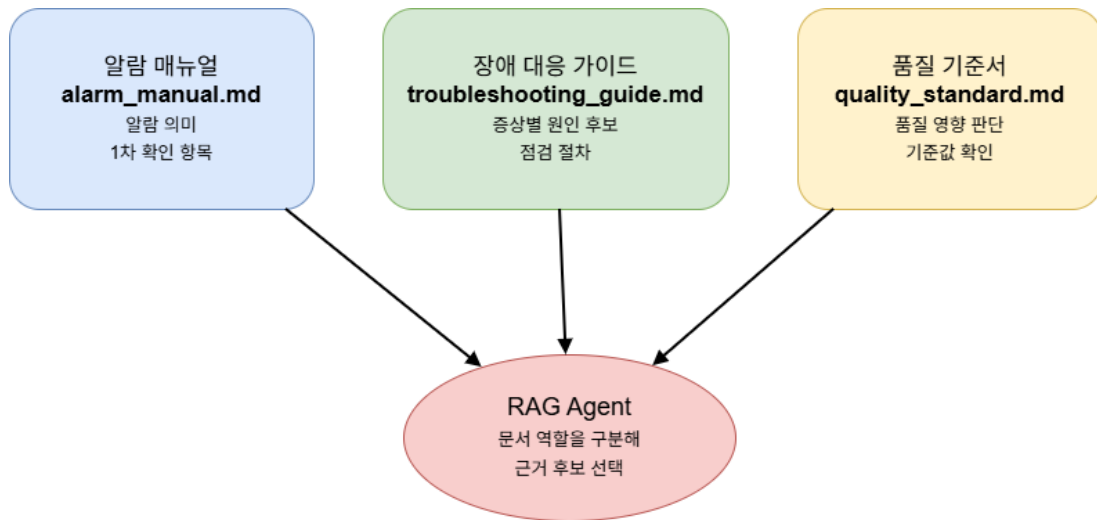
불량률, 수율, 검사 기준 확인

정비 절차서

설비 점검과 교체 이력 이해

각 문서는 역할이 다릅니다. Agent는 여러 문서에서 근거 후보를 수집하고, 각 문서의 역할에 따라 답변 근거를 분리해 사용해야 합니다.

문서별 역할 구분



정비 절차서는 Day2 기본 문서가 아니라 확장 예시로 구분합니다.

실습 범위: 교육용 가상 시나리오 사용

이번 실습에서 사용하지 않는 것

- 실제 사내 내부 시스템명
- 실제 설비 운영 데이터
- 실제 API Key
- 실제 제조 기 내부 문서

교육용 가상 시나리오

LINE-07 · EQP-EV-03 · ALM-TEMP-402

docs 폴더의 Markdown 파일을 RAG 검색 대상 문서로 사용합니다.

 Markdown 형식은 문서 내용을 읽고, 문단 단위로 분리하고, 검색 대상으로 변환하기 쉬워 교육용 RAG 실습에 적합합니다.

이번 실습의 목표는 제조 기술 문서 검색 → 근거 추출 → 답변 생성 → Trace 확인 흐름을 안전한 교육용 환경에서 연습하는 것입니다.

RAG 전체 실행 흐름

RAG는 문서를 읽고 답변하는 단순 기능이 아닙니다. 문서를 검색 가능한 구조로 만들고, 검색 결과를 State와 Node 흐름에 연결하여 검토 가능한 답변을 생성하는 구조입니다.

01	02
문서 로드 Markdown 문서를 시스템에 로드	Chunk 생성 문서를 검색 단위로 분할
03	04
Metadata 생성 각 Chunk에 메타데이터 추가	Top-3 검색 질문 관련 근거 상위 3개 추출
05	06
State 저장 검색 결과를 State와 연동	답변 생성 Node 흐름으로 응답 초안 생성
07	08
근거 검증 생성된 답변의 근거 검토	Trace 확인 처리 이력과 근거 추적 확인

단계	역할
Markdown 문서 로드	RAG 대상 문서 입력 구조 준비
Chunk / Metadata 생성	문서를 검색 가능한 지식 단위로 변환
Top-3 검색	질문과 관련 있는 근거 후보 추출
State 저장 → Node 실행	검색 결과를 Node 사이에서 공유되는 업무 처리 문맥으로 전달
근거 검증 → Trace 확인	Grounding 검증 및 실행 흐름 검토

2일차 기본 범위와 선택 실습 구분

2일차 기본 범위

- Markdown 문서 로드
- Chunk 생성 및 Metadata 생성
- Top-3 검색
- State 저장 및 Node 흐름
- 근거 검증 및 Trace 확인
- RAG Agent v1 실행

선택 / 확장 실습

- Chroma 구축 세부 설정
- Embedding 수식 이해
- Ollama 설치 문제 해결
- Vector DB 비교 분석
- Streamlit 보조 UI 활용

 Chroma/Ollama 실행 환경 문제로 시간이 오래 소요될 경우, 실시간 디버깅을 길게 하지 않고 **outputs/day2**에 준비된 실행 결과를 기준으로 **Saved Result Review** 방식으로 진행합니다.

오늘의 기본 목표는 Vector DB 구축 자체가 아니라, 검색 결과가 State와 Node 흐름을 거쳐 답변 근거로 사용되는 구조를 이해하는 것입니다.

2.2일차 실습 프로젝트 구조와 실행 흐름

RAG Agent v1 프로젝트 구조와 폴더 역할

폴더 구조는 단순 정리용이 아닙니다. 입력 문서, 실행 코드, 템플릿, 산출물, Trace를 분리해 Agent 실행 흐름을 추적하기 위한 아키텍처 구조입니다.

폴더	역할	수강생이 확인할 내용
docs/	RAG 대상 교육용 문서	알람 매뉴얼, 장애 대응 가이드, 품질 기준서
data/	표준 입력 사례	RAG 검색 질문과 대표 장애 질의
templates/day2/	결과 문서 템플릿	실행 결과를 Markdown으로 저장할 때 사용하는 틀
src/day2/	2일차 RAG 실행 파일	문서 로드, chunk, 검색, State, Graph, 최종 실행 파일
outputs/day2/	실행 결과와 Trace	검색 결과, State 변화, Trace, 최종 결과

확장 문서 구분

❏ docs 폴더에는 2일차 RAG 실습에 사용할 교육용 Markdown 문서가 있습니다. 이 세 문서는 Agent가 답변 근거를 검색할 교육용 지식 소스입니다.

alarm_manual. md

알람 코드의 의미와 1차 확인 항목을 설명합니다. ALM-TEMP-402 등 교육용 알람 코드 기반으로 구성됩니다.

troubleshooting _guide.md

장애 증상별 원인 후보와 점검 절차를 담은 문서입니다. 제조 장애 대응 흐름을 재현합니다.

quality_standard. md

품질 기준과 품질 이상 판단 기준을 설명합니다. 불량률, 수율, 검사 기준 확인에 사용됩니다.

❗ 실제 사내 문서는 아니지만, 제조 장애 대응 Agent가 알람 매뉴얼·장애 대응 가이드·품질 기준서를 함께 참조하는 구조를 안전하게 재현합니다.

data 폴더: 표준 입력 사례

data 폴더의 파일은 수강생이 직접 긴 질문을 만들기 전에 사용할 수 있는 표준 입력 사례입니다. 동일한 입력을 사용하면 검색 결과, State 변화, 최종 답변 결과를 비교하기 쉽습니다.

파일명	역할	실습에서 사용하는 위치
sample_rag_queries.json	RAG 검색 테스트용 질문 목록	rag_search.py 실행 시 검색 질문 예시로 사용
sample_query.json	대표 장애 질문 입력	day2_rag_agent_v1.py 실행 흐름에서 사용자 질문 예시로 사용

 data 폴더는 RAG 검색과 최종 Agent 실행을 재현 가능한 기준 입력으로 관리하는 위치입니다. 동일 입력 기반의 결과 비교가 검색 품질 검토의 출발점입니다.

src/day2 실행 파일 구조

파일을 하나씩 읽는 것이 아니라, 문서 준비 → 지식 단위 생성 → 근거 검색 → State 저장 → Node 실행 → 최종 Agent 결과 생성 흐름으로 묶어 확인합니다.

파일명	역할
rag_document_loader.py	RAG 대상 Markdown 문서 로드
chunk_builder.py	문서 chunk와 metadata 생성
rag_search.py	관련 문서 Top-3 검색
graph_state.py	Agent State 구조 확인
langgraph_rag_graph_runner.py	검색·답변·근거 검증 Node 흐름
day2_rag_agent_v1.py	Day2 RAG Agent v1 최종 실행
chroma_index_builder.py	Chroma 인덱스 생성
day2_rag_agent_streamlit_app.py	Streamlit 보조 화면

⚠ rag_search.py는 필수 흐름에 포함되지만, Chroma/Ollama 환경에 따라 실행이 지연될 수 있습니다. 이 경우 outputs/day2/rag_test_results.md를 기준으로 검색 결과 해석을 진행합니다.

outputs/day2 산출물 구조

outputs/day2 폴더는 실행 결과뿐 아니라 검색 근거, State 변화, Trace, 최종 답변을 검토하는 기준 자료가 모이는 위치입니다.

산출물 파일	생성 시점	의미
document_load_result.md	문서 로드 후	RAG 대상 문서 입력 구조 확인
chunk_preview.json	chunk 생성 후	문서 조각과 metadata 미리보기
chunk_build_result.md	chunk 생성 후	chunk 생성 요약 및 검토
rag_test_results.md	RAG 검색 후	Top-3 검색 결과와 근거 후보 확인
state_demo_result.md	State 실행 후	검색 전후 State 변화 확인
langgraph_rag_trace.md	RAG Graph 실행 후	Node 실행 순서, State 변화, 근거 검증 결과 Trace
day2_rag_agent_v1_result.md	최종 실행 후	RAG Agent v1 최종 결과와 근거 기반 답변 초안
chroma_build_result.md	선택 실습 후	Chroma 인덱스 생성 결과

 산출물은 단순 결과물이 아니라 RAG Agent v1이 어떤 문서를 검색하고 어떤 흐름으로 답변을 생성했는지 확인하는 Trace 기반 검토 자료입니다.

3. Markdown 문서 로드와 지식 구조 준비

rag_document_loader.py 역할

src/day2/rag_document_loader.py는 docs 폴더의 Markdown 문서를 읽어 RAG 검색 대상으로 준비하는 실습 파일입니다. 이 파일은 문서를 바로 답변에 사용하는 것이 아니라, 이후 chunk 생성과 검색에 사용할 원천 문서로 준비합니다.

읽는 문서

- docs/alarm_manual.md
- docs/troubleshooting_guide.md
- docs/quality_standard.md

설계 관점

Markdown 문서는 문단 구조를 유지하면서도 Python에서 읽고 분리하기 쉬워 교육용 RAG 문서로 적합합니다.

알람 매뉴얼, 장애 대응 가이드, 품질 기준서는 모두 RAG 검색의 근거 문서가 됩니다.

 이 파일은 2일차 RAG 실습에서 문서 입력 구조를 준비하는 출발점입니다.

실행 명령어: 문서 로드

명령어는 manufacturing_agent_project 폴더의 루트 위치에서 실행합니다. 루트 위치는 src 폴더와 docs 폴더가 보이는 위치입니다.

```
uv run python src/day2/rag_document_loader.py
```

01

PowerShell 실행

PowerShell을 열고 프로젝트 루트 폴더로 이동합니다.

02

명령어 실행

위 명령어를 입력하여 교육용 Markdown 문서를 로드합니다.

03

결과 확인

outputs/day2 폴더에 document_load_result.md가 생성되었는지 확인합니다.



이 실행을 통해 RAG Agent가 사용할 문서 입력이 정상적으로 준비되었는지 확인합니다.

예상 출력 해석

실행 결과에서 확인하는 항목은 단순 통계가 아닙니다. 이후 **chunk** 생성 전략을 결정하는 기초 정보입니다.

문서 파일명

읽은 문서의 이름 — RAG 검색 대상 범위 확인

글자 수

문서 안에 포함된 텍스트 규모 — chunk 크기 설계 기준

문단 수

이후 chunk 생성 시 문서를 어떤 단위로 나눌 수 있는지 판단하는 기준

로드 성공 여부

문서를 정상적으로 읽었는지 확인 — 입력 구조 검증

 문서가 너무 길거나 문단 수가 많으면 검색 단위를 어떻게 나눌지 설계해야 합니다. 출력 결과를 보고 chunk 설계 방향을 미리 판단하세요.

산출물 확인: document_load_result.md

확인 위치

outputs/day2/document_load_result.md

이 파일은 문서 로드 결과를 사람이 읽기 좋게 정리한
Markdown 산출물입니다.

- 어떤 문서를 읽었는지 확인
- 문서 로드가 정상적으로 되었는지 확인
- 다음 단계 chunk 생성 실습 진행 전 입력 문서 준비
상태 검증

설계 관점

document_load_result.md는 RAG 검색 대상으로 사용할 문서가 정상적으로 준비되었는지 확인하는 입력 검증 자료입니다.

이 검증 없이 chunk 생성으로 넘어가면 잘못된 문서 범위로 검색 구조가 구성될 수 있습니다.

결과 해석과 다음 단계 연결

이제 Agent가 검색할 문서를 읽었습니다. 하지만 긴 문서 전체를 그대로 검색 대상으로 사용하면 두 가지 문제가 발생합니다.

문제 1: 관련 없는 문맥 포함

긴 문서 전체를 검색 결과로 가져오면 어느 부분이 중요한지 알기 어렵습니다.

문제 2: 근거 초점 흐림

LLM에게 전달되는 근거가 너무 넓어져 답변의 초점이 흐려질 수 있습니다.

- ✔ 따라서 다음 단계에서는 문서를 검색과 근거 제시에 적합한 단위로 나눕니다. 이 단위를 **chunk**라고 합니다. 검색 품질과 근거 추적을 높이기 위해 문서를 적절한 chunk 단위로 나누어야 합니다.

4.chunk 생성과 검색 단위 설계

왜 문서를 적절한 단위로 나누어야 하는가

근거 정확성 향상

문단 단위 chunk를 사용하면 LLM에게 필요한 근거만 전달하기 쉽습니다. 문서 전체를 가져오면 어느 부분이 중요한지 알기 어렵습니다.

Trace 검토 가능성

검색 결과에서 어떤 문서의 어떤 부분이 근거로 사용되었는지 더 명확히 확인할 수 있습니다.

검색 효율성

문서가 길어져도 필요한 부분을 빠르게 찾을 수 있습니다. 검색 속도와 정확도가 함께 향상됩니다.

❏ chunk는 검색 속도뿐 아니라 답변 근거의 정확성, 문맥 유지, Trace 검토 가능성에 영향을 주는 핵심 설계 요소입니다.

chunk_builder.py 역할

src/day2/chunk_builder.py는 Markdown 문서에서 문단을 읽고, 문서를 검색 가능한 chunk로 나누는 파일입니다. 각 chunk에는 식별 정보가 부여됩니다.

chunk에 부여되는 식별 정보

chunk_id: 문서 조각을 식별하기 위한 값. 검색 결과와 Trace에서 근거 위치를 추적하는 기준.

doc_name: 이 문서 조각이 어느 원본 문서에서 왔는지 알려주는 출처 정보.

설계 관점

너무 짧은 문단은 검색 근거로 쓰기 어려울 수 있어 제외될 수 있습니다.

chunk_id와 doc_name은 이후 rag_test_results.md와 langgraph_rag_trace.md에서도 근거 확인에 사용됩니다.

 이 파일은 문서를 검색 가능한 지식 단위로 변환하고, 검색 근거를 추적할 수 있도록 식별 정보를 부여하는 역할을 합니다.

실행 명령어: Chunk 생성

chunk 생성도 프로젝트 루트 위치에서 실행합니다. src 폴더와 docs 폴더가 보이는 위치에서 아래 명령어를 실행합니다.

```
uv run python src/day2/chunk_builder.py
```

01

문서 읽기

docs 폴더의 Markdown 문서를 읽습니다.

03

식별 정보 부여

각 chunk에 chunk_id와 doc_name을 부여합니다.

02

Chunk 분리

문단 단위로 검색하기 좋은 chunk로 나눕니다.

04

산출물 저장

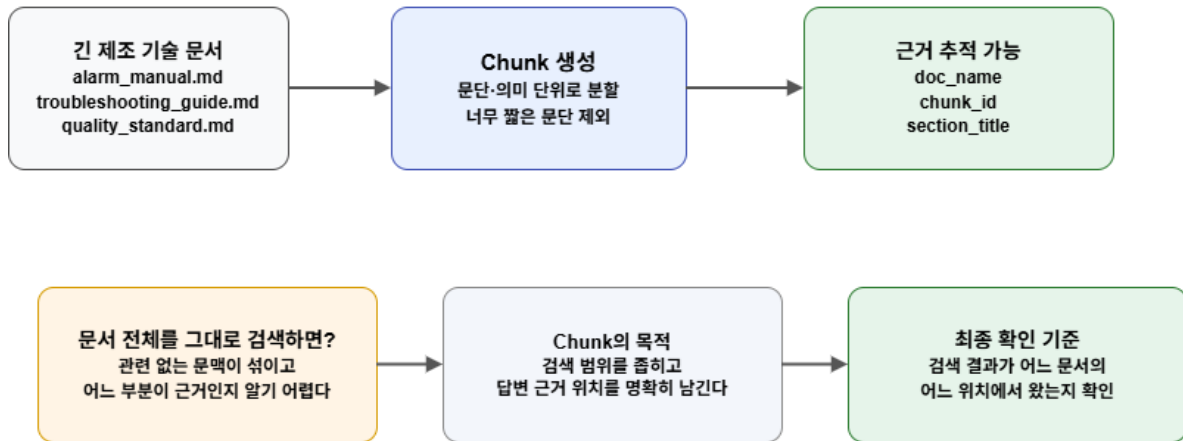
outputs/day2 폴더에서 chunk 관련 산출물을 확인합니다.



이 실행을 통해 제조 기술 문서를 Agent가 검색하고 근거로 제시할 수 있는 지식 단위로 변환합니다.

Chunk의 설계 의미

문서 분할이 아니라, 검색 근거를 추적하기 위한 단위 설계



핵심 문장: Chunk는 단순 분할 결과가 아니라, RAG 답변의 출처를 설명하기 위한 근거 단위입니다.

Chunk 생성 기준: 설계 관점

chunk 생성 기준은 문서를 단순히 잘게 나누는 규칙이 아니라, 검색 품질과 근거 추적 가능성을 높이기 위한 설계 기준입니다.

기준	의미	설계 관점
문단 단위	문서를 문단별로 나눔	알람 의미, 증상, 조치 절차가 끊기지 않도록 의미 단위를 유지
너무 짧은 문단 제외	근거로 부족한 짧은 내용은 제외	제목만 있는 문단은 답변 근거로 사용하기 어려움
chunk_id	문서 조각 식별자	검색 결과와 Trace에서 근거 위치를 추적하는 기준
doc_name	원본 문서명	어떤 문서를 근거로 사용했는지 확인하는 기준

산출물 확인: chunk_preview.json & chunk_build_result.md

chunk_preview.json

이후 검색 로직이 재사용할 구조화된 chunk 데이터입니다.

- 문서 조각 (chunk 텍스트)
- 원본 문서명 (doc_name)
- chunk 식별자 (chunk_id)
- metadata 정보

chunk_build_result.md

수강생과 강사가 chunk 생성 결과를 검토하기 위한 요약 문서입니다.

- 어떤 문서에서 몇 개의 chunk가 생성되었는지 확인
- 생성 결과가 정상적인지 확인
- 검색 품질 검토 자료로 활용

□ chunk_preview.json은 검색 실습에서 재사용할 구조화 데이터이고, chunk_build_result.md는 생성 결과를 검토하기 위한 검색 품질 검토 자료입니다.

결과 해석과 다음 단계: Metadata 설계

이제 긴 Markdown 문서를 chunk로 나누었습니다. 하지만 chunk만으로는 각 문서 조각이 어떤 알람 코드, 설비, 증상, 조치 항목과 연결되는지 알기 어렵습니다.

Metadata가 필요한 이유

각 chunk에 검색과 필터링에 사용할 구조화 정보를 붙여야 합니다.

Metadata 예시

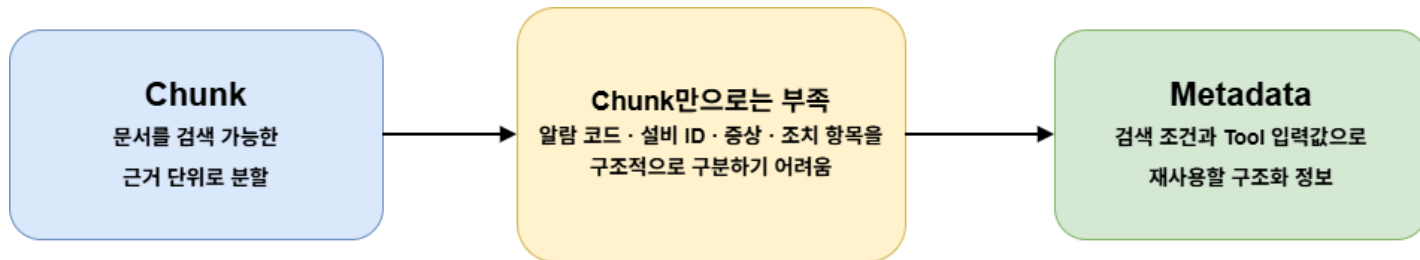
ALM-TEMP-402 · EQP-EV-03 ·
온도 편차 · 품질 영향

다음 단계 연결

metadata는 검색 품질과 3일차
search_manual MCP Tool 입력값
설계에 직접 연결됩니다.

- ✔ 문서 조각을 만들었다면 다음 장에서는 metadata가 검색 품질과 3일차 Tool 입력값 설계에 어떻게 연결되는지 확인합니다. 이 구조는 4일차 Trace 기반 품질 평가의 기반이 됩니다.

Chunk에서 Metadata로 넘어가는 이유



Metadata 예시: alarm_code · equipment_id · symptom · action · keywords

핵심: Chunk는 근거 위치를 추적하는 단위이고, Metadata는 검색과 Tool 연계를 위한 구조화 기준입니다.